

Coding & Solution

Date	1 July 2026
Team ID	—
Project Name	Rising Waters – Flood Prediction System
Maximum Marks	5 Marks

This section evaluates the quality and completeness of the implemented code and the overall Rising Waters solution.

Solution Summary

Field	Details
Repository Link / URL	[Add your GitHub/GitLab repository link here before submission]
Programming Language(s)	Python 3
Framework(s) Used	Flask (backend), scikit-learn & XGBoost (ML), pandas/numpy (data), matplotlib/seaborn (visualization)
Key Features Implemented	Dataset generation & EDA scripts; preprocessing (missing values, outlier capping); multi-model training with best-model auto-selection; Flask web app with prediction form and risk-specific result pages
Pending / Incomplete Features	Cloud deployment (documented as an optional next step, not yet executed); user accounts / history tracking
Setup / Run Instructions	pip install -r requirements.txt → python notebooks/0_generate_dataset.py → python notebooks/2_preprocessing_and_training.py → python app.py → open http://127.0.0.1:5000/

Code Quality Checklist

S.No	Criteria	Status (Yes / No)
1	Code is modular and organized into functions / classes	Yes
2	Meaningful variable and function names are used	Yes
3	Code includes comments / documentation where necessary	Yes
4	Error handling is implemented for critical operations	Yes
5	The application runs without critical errors	Yes
6	Code is committed to a version control repository	Yes

Additional Notes / Comments: The training script automatically falls back to scikit-learn's GradientBoostingClassifier if the xgboost package isn't available, so the pipeline always runs end-to-end even in restricted environments.

Key Code Snippet — Flask Prediction Route (app.py)

```
@app.route("/predict", methods=["POST"])
def predict():
    try:
        input_values = []
        readable_inputs = {}
        for name in FEATURE_NAMES:
            raw = request.form.get(name, FEATURE_CONFIG.get(name,
{}).get("default", 0))
            val = float(raw)
            input_values.append(val)
            readable_inputs[FEATURE_CONFIG.get(name, {}).get("label", name)] = val

        X = pd.DataFrame([input_values], columns=FEATURE_NAMES)
        X_scaled = scaler.transform(X)
        prediction = int(model.predict(X_scaled)[0])
        ...
    except Exception as e:
        return render_template("predict.html", fields=fields, error=str(e))
```